Lambda Calculus

FOCS, Fall 2020

Computation as algebra

Turing machines and production grammars are fundamentally computer science models — they can be used to define computation as decision problems over strings of symbols

This week - a model of computation originally due to logicians (Church 1936) to capture computability for natural number functions $N^k \rightarrow N$

Intuition: to compute is to simplify an algebraic expression until you get a result

Expressions of the λ calculus

λ calculus expressions (also called terms) are one of the following:

- identifiers (x, y, z, …)
- abstractions <x → M> where x is an identifier and M is a term
- applications (M N) where M, N are terms

We assume that application associates to the left, so that we can drop parentheses around applications in some situations, so that M N P = (M N) P

Simplification rule

A single simplification rule captures how abstraction interacts with application

$$(\langle x \rightarrow M \rangle\ N) = M\{^{N}/x\}$$

where $M\{^{N}/x\}$ is M in which every *free occurrence* of x is replaced by N

- A free occurrence of x is a use of x that is not in the body of any $\langle x \rightarrow N \rangle$
- Need to make sure free occurrences of identifiers in N are not *captured*:
  - rename non-free occurrences in M to new identifiers if needed

We are allowed to simplify anywhere in an expression

Simplification rule

A single simplification rule captures

$(\langle x \to M\rangle\ N) = M\{^N/x\}$

where $M\{^N/x\}$ is M in which every *fr*

- A free occurrence of x is a use
- Need to make sure free occurr
  - rename non-free occurrences in M

We are allowed to simplify anywher

One x is free in $\langle y \to x\ y\rangle$

One x is free in $\langle y \to x\ \langle x \to y\rangle\rangle$

Two x's are free in

$\langle y \to \langle z \to x\ \langle x \to x\ x\rangle\ x\rangle\ y\rangle$

Two x's and one y are free in

$\langle y \to \langle z \to x\ \langle x \to x\ x\rangle\ x\rangle\rangle\ y$

No identifier is free in:

$\langle x \to \langle y \to \langle z \to x\ x\ y\rangle\rangle\rangle$

Examples

$\langle x \to x \rangle \langle y \to y \rangle = \langle y \to y \rangle$

Examples

$\langle x \to x \rangle \langle y \to y \rangle \quad = \quad \langle y \to y \rangle$

$(\langle x \to \langle y \to x\ y \rangle\rangle\ z_1)\ z_2 \quad = \langle y \to z_1\ y \rangle\ z_2$

Examples

$$\langle x \to x \rangle \langle y \to y \rangle \quad = \quad \langle y \to y \rangle$$

$$(\langle x \to \langle y \to x\ y \rangle\rangle\ z_1)\ z_2 \quad = \langle y \to z_1\ y \rangle\ z_2$$

$$= z_1\ z_2$$

Examples

$\langle x \to x\rangle \langle y \to y\rangle \;=\; \langle y \to y\rangle$

$(\langle x \to \langle y \to x\ y\rangle\rangle\ z_1)\ z_2 \;=\; \langle y \to z_1\ y\rangle\ z_2$

$= z_1\ z_2$

$(\langle x \to \langle y \to x\ y\rangle\rangle\ \langle z \to z\rangle)\ \langle x \to \langle y \to x\rangle\rangle \;=\; \langle y \to \langle z \to z\rangle\ y\rangle\ \langle x \to \langle y \to x\rangle\rangle$

Examples

$$\langle x \to x\rangle\ \langle y \to y\rangle \quad = \quad \langle y \to y\rangle$$

$$(\langle x \to \langle y \to x\ y\rangle\rangle\ z_1)\ z_2 \quad = \langle y \to z_1\ y\rangle\ z_2$$
$$= z_1\ z_2$$

$$(\langle x \to \langle y \to x\ y\rangle\rangle\ \langle z \to z\rangle)\ \langle x \to \langle y \to x\rangle\rangle \quad = \langle y \to \langle z \to z\rangle\ y\rangle\ \langle x \to \langle y \to x\rangle\rangle$$
$$= \langle z \to z\rangle\ \langle x \to \langle y \to x\rangle\rangle$$

Examples

$$\langle x \to x\rangle\ \langle y \to y\rangle \quad = \quad \langle y \to y\rangle$$

$$(\langle x \to \langle y \to x\ y\rangle\rangle\ z_1)\ z_2 \quad = \langle y \to z_1\ y\rangle\ z_2$$
$$= z_1\ z_2$$

$$(\langle x \to \langle y \to x\ y\rangle\rangle\ \langle z \to z\rangle)\ \langle x \to \langle y \to x\rangle\rangle \quad = \langle y \to \langle z \to z\rangle\ y\rangle\ \langle x \to \langle y \to x\rangle\rangle$$
$$= \langle z \to z\rangle\ \langle x \to \langle y \to x\rangle\rangle$$
$$= \langle x \to \langle y \to x\rangle\rangle$$

Examples

$$\langle x \to x\rangle\ \langle y \to y\rangle \quad = \quad \langle y \to y\rangle$$

$$(\langle x \to \langle y \to x\ y\rangle\rangle\ z_1)\ z_2 \quad = \langle y \to z_1\ y\rangle\ z_2$$
$$= z_1\ z_2$$

$$(\langle x \to \langle y \to x\ y\rangle\rangle\ \langle z \to z\rangle)\ \langle x \to \langle y \to x\rangle\rangle \quad = \langle y \to \langle z \to z\rangle\ y\rangle\ \langle x \to \langle y \to x\rangle\rangle$$
$$= \langle z \to z\rangle\ \langle x \to \langle y \to x\rangle\rangle$$
$$= \langle x \to \langle y \to x\rangle\rangle$$

$$(\langle x \to \langle y \to y\ x\rangle\rangle\ \langle z \to z\rangle)\ \langle x \to \langle y \to x\rangle\rangle \quad = \langle y \to y\ \langle z \to z\rangle\rangle\ \langle x \to \langle y \to x\rangle\rangle$$

Examples

$$\langle x \to x\rangle\ \langle y \to y\rangle \quad = \quad \langle y \to y\rangle$$

$$(\langle x \to \langle y \to x\ y\rangle\rangle\ z_1)\ z_2 \quad = \langle y \to z_1\ y\rangle\ z_2$$
$$= z_1\ z_2$$

$$(\langle x \to \langle y \to x\ y\rangle\rangle\ \langle z \to z\rangle)\ \langle x \to \langle y \to x\rangle\rangle \quad = \langle y \to \langle z \to z\rangle\ y\rangle\ \langle x \to \langle y \to x\rangle\rangle$$
$$= \langle z \to z\rangle\ \langle x \to \langle y \to x\rangle\rangle$$
$$= \langle x \to \langle y \to x\rangle\rangle$$

$$(\langle x \to \langle y \to y\ x\rangle\rangle\ \langle z \to z\rangle)\ \langle x \to \langle y \to x\rangle\rangle \quad = \langle y \to y\ \langle z \to z\rangle\rangle\ \langle x \to \langle y \to x\rangle\rangle$$
$$= \langle x \to \langle y \to x\rangle\rangle\ \langle z \to z\rangle$$

Examples

$$\langle x \to x\rangle\ \langle y \to y\rangle \quad = \quad \langle y \to y\rangle$$

$$(\langle x \to \langle y \to x\ y\rangle\rangle\ z_1)\ z_2 \quad = \langle y \to z_1\ y\rangle\ z_2$$
$$= z_1\ z_2$$

$$(\langle x \to \langle y \to x\ y\rangle\rangle\ \langle z \to z\rangle)\ \langle x \to \langle y \to x\rangle\rangle = \langle y \to \langle z \to z\rangle\ y\rangle\ \langle x \to \langle y \to x\rangle\rangle$$
$$= \langle z \to z\rangle\ \langle x \to \langle y \to x\rangle\rangle$$
$$= \langle x \to \langle y \to x\rangle\rangle$$

$$(\langle x \to \langle y \to y\ x\rangle\rangle\ \langle z \to z\rangle)\ \langle x \to \langle y \to x\rangle\rangle = \langle y \to y\ \langle z \to z\rangle\rangle\ \langle x \to \langle y \to x\rangle\rangle$$
$$= \langle x \to \langle y \to x\rangle\rangle\ \langle z \to z\rangle$$
$$= \langle y \to \langle z \to z\rangle\rangle$$

Normal forms

An expression is in *normal form* if no simplification is possible

- it has no subexpression of the form $\langle x \to M \rangle N$

An expression has a normal form if you can simplify it to a normal form

You can think of an expression in normal form as a result

Existence of normal forms

Not every expression has a normal form

$\langle x \to x\ x \rangle\ \langle x \to x\ x \rangle$

Existence of normal forms

Not every expression has a normal form

$\langle x \to x\ x \rangle\ \langle x \to x\ x \rangle = \langle x \to x\ x \rangle\ \langle x \to x\ x \rangle$

Existence of normal forms

Not every expression has a normal form

$$\langle x \to x\ x\rangle\ \langle x \to x\ x\rangle = \langle x \to x\ x\rangle\ \langle x \to x\ x\rangle$$
$$= \langle x \to x\ x\rangle\ \langle x \to x\ x\rangle$$

Existence of normal forms

Not every expression has a normal form

$$
\begin{aligned}
\langle x \to x\ x\rangle\ \langle x \to x\ x\rangle &= \langle x \to x\ x\rangle\ \langle x \to x\ x\rangle \\
&= \langle x \to x\ x\rangle\ \langle x \to x\ x\rangle \\
&= \langle x \to x\ x\rangle\ \langle x \to x\ x\rangle
\end{aligned}
$$

Existence of normal forms

Not every expression has a normal form

$$
\begin{aligned}
\langle x \to x\ x\rangle\ \langle x \to x\ x\rangle &= \langle x \to x\ x\rangle\ \langle x \to x\ x\rangle \\
&= \langle x \to x\ x\rangle\ \langle x \to x\ x\rangle \\
&= \langle x \to x\ x\rangle\ \langle x \to x\ x\rangle \\
&= \ldots
\end{aligned}
$$

Church-Rosser Theorem: If an expression has a normal form, every simplification that yields a normal form yields the same normal form

In English: order of simplification is irrelevant, but some orders may not terminate

Abbreviations

I'm generally going to write:

$\langle x\ y \to M\rangle$ for $\langle x \to \langle y \to M\rangle\rangle$

$\langle x\ y\ z \to M\rangle$ for $\langle x \to \langle y \to \langle z \to M\rangle\rangle\rangle$

...

So that for example:

$\langle x\ y \to M\rangle\ N = \langle x \to \langle y \to M\rangle\rangle\ N$

$= \langle y \to M\{^N/x\}\rangle$

$\langle x\ y\ z \to M\rangle\ N = \langle x \to \langle y \to \langle z \to M\rangle\rangle\rangle\ N$

$= \langle y \to \langle z \to M\{^N/x\}\rangle\rangle$

$= \langle y\ z \to M\{^N/x\}\rangle$

Encoding Booleans

The lambda calculus only has identifiers and functions - no other "types" of values

We can encode other types of values - if it looks, swims, and quacks like a duck…

Encoding Booleans

The lambda calculus only has identifiers and functions - no other "types" of values

We can encode other types of values - if it looks, swims, and quacks like a duck…

true = <x y → x> (= <x → <y → x>>)

false = <x y → y> (= <x → <y → y>>)

Encoding Booleans

The lambda calculus only has identifiers and functions - no other "types" of values

We can encode other types of values - if it looks, swims, and quacks like a duck…

true = <x y → x> (= <x → <y → x>>)

false = <x y → y> (= <x → <y → y>>)

if = <c t e → (c t) e> (= <c → <t → <e → (c t) e>>>)

Encoding Booleans

The lambda calculus only has identifiers and functions - no other "types" of values

We can encode other types of values - if it looks, swims, and quacks like a duck…

true = <x y → x> (= <x → <y → x>>)

false = <x y → y> (= <x → <y → y>>)

if = <c t e → (c t) e> (= <c → <t → <e → (c t) e>>>)

Properties desired: **if true** z_1 z_2 = z_1
if false z_1 z_2 = z_2

Encoding Booleans

The lambda calculus only has identifiers a

We can encode other types of values - if i

true = <x y → x> (= <x → <y → x>>

false = <x y → y> (= <x → <y → y>>

if = <c t e → (c t) e> (= <c → <t → <e →

Properties desired: **if true** z_1 z_2 = z_1
if false z_1 z_2 = z_2

if true z_1 z_2
= <c t e → (c t) e> **true** z_1 z_2
= <t e → (**true** t) e> z_1 z_2
= <e → (**true** z_1) e> z_2
= (**true** z_1) z_2
= (<x y → x> z_1) z_2
= <y → z_1> z_2
= z_1

Encoding Booleans

The lambda calculus only has identifiers a

We can encode other types of values - if i

true = <x y → x> (= <x → <y → x>>

false = <x y → y> (= <x → <y → y>>

if = <c t e → (c t) e> (= <c → <t → <e →

Properties desired: **if true** z_1 z_2 = z_1
if false z_1 z_2 = z_2

if false z_1 z_2
= <c t e → (c t) e> **false** z_1 z_2
= <t e → (**false** t) e> z_1 z_2
= <e → (**false** z_1) e> z_2
= (**false** z_1) z_2
= (<x y → y> z_1) z_2
= <y → y> z_2
= z_2

Encoding pairs

A pair is a data structure holding two values, from which you can extract out either

Encoding pairs

A pair is a data structure holding two values, from which you can extract out either

pair = <x y → <s → (s x) y>>

Encoding pairs

A pair is a data structure holding two values, from which you can extract out either

pair = <x y → <s → (s x) y>>

fst = <p → p <x y → x>>

snd = <p → p <x y → y>>

Encoding pairs

A pair is a data structure holding two values, from which you can extract out either

pair = <x y → <s → (s x) y>>

fst = <p → p <x y → x>>

snd = <p → p <x y → y>>

Properties desired: **fst** (**pair** z_1 z_2) = z_1
snd (**pair** z_1 z_2) = z_2

Encoding pairs

A pair is a data structure holding two val

pair = <x y → <s → (s x) y>>

fst = <p → p <x y → x>>

snd = <p → p <x y → y>>

Properties desired: **fst** (**pair** z_1 z_2) = z

snd (**pair** z_1 z_2) =

fst (**pair** z_1 z_2)

Encoding pairs

A pair is a data structure holding two val

pair = <x y → <s → (s x) y>>

fst = <p → p <x y → x>>

snd = <p → p <x y → y>>

Properties desired: **fst** (**pair** z_1 z_2) = z
snd (**pair** z_1 z_2) =

fst (**pair** z_1 z_2)
= **fst** (<x y → <s → (s x) y>> z_1 z_2)

Encoding pairs

A pair is a data structure holding two val

pair = <x y → <s → (s x) y>>

fst = <p → p <x y → x>>

snd = <p → p <x y → y>>

Properties desired: **fst** (**pair** z_1 z_2) = z
snd (**pair** z_1 z_2) =

fst (**pair** z_1 z_2)
= **fst** (<x y → <s → (s x) y>> z_1 z_2)
= **fst** (<y → <s → (s z_1) y>> z_2)

Encoding pairs

A pair is a data structure holding two val

pair = <x y → <s → (s x) y>>

fst = <p → p <x y → x>>

snd = <p → p <x y → y>>

Properties desired: **fst** (**pair** z_1 z_2) = z
snd (**pair** z_1 z_2) =

fst (**pair** z_1 z_2)
= **fst** (<x y → <s → (s x) y>> z_1 z_2)
= **fst** (<y → <s → (s z_1) y>> z_2)
= **fst** <s → (s z_1) z_2>

Encoding pairs

A pair is a data structure holding two val

pair = <x y → <s → (s x) y>>

fst = <p → p <x y → x>>

snd = <p → p <x y → y>>

Properties desired: **fst** (**pair** z_1 z_2) = z

snd (**pair** z_1 z_2) =

fst (**pair** z_1 z_2)

= **fst** (<x y → <s → (s x) y>> z_1 z_2)

= **fst** (<y → <s → (s z_1) y>> z_2)

= **fst** <s → (s z_1) z_2>

= <p → p <x y → x>> <s → (s z_1) z_2>

Encoding pairs

A pair is a data structure holding two val

pair = <x y → <s → (s x) y>>

fst = <p → p <x y → x>>

snd = <p → p <x y → y>>

Properties desired: **fst** (**pair** z_1 z_2) = z
snd (**pair** z_1 z_2) =

fst (**pair** z_1 z_2)
= **fst** (<x y → <s → (s x) y>> z_1 z_2)
= **fst** (<y → <s → (s z_1) y>> z_2)
= **fst** <s → (s z_1) z_2>
= <p → p <x y → x>> <s → (s z_1) z_2>
= <s → (s z_1) z_2> <x y → x>

Encoding pairs

A pair is a data structure holding two val

pair = <x y → <s → (s x) y>>

fst = <p → p <x y → x>>

snd = <p → p <x y → y>>

Properties desired: **fst** (**pair** z_1 z_2) = z

snd (**pair** z_1 z_2) =

fst (**pair** z_1 z_2)

= **fst** (<x y → <s → (s x) y>> z_1 z_2)

= **fst** (<y → <s → (s z_1) y>> z_2)

= **fst** <s → (s z_1) z_2>

= <p → p <x y → x>> <s → (s z_1) z_2>

= <s → (s z_1) z_2> <x y → x>

= (<x y → x> z_1) z_2

Encoding pairs

A pair is a data structure holding two val

pair = <x y → <s → (s x) y>>

fst = <p → p <x y → x>>

snd = <p → p <x y → y>>

Properties desired: **fst** (**pair** z_1 z_2) = z

snd (**pair** z_1 z_2) =

fst (**pair** z_1 z_2)

= **fst** (<x y → <s → (s x) y>> z_1 z_2)

= **fst** (<y → <s → (s z_1) y>> z_2)

= **fst** <s → (s z_1) z_2>

= <p → p <x y → x>> <s → (s z_1) z_2>

= <s → (s z_1) z_2> <x y → x>

= (<x y → x> z_1) z_2

= <y → z_1> z_2

Encoding pairs

A pair is a data structure holding two val

pair = <x y → <s → (s x) y>>

fst = <p → p <x y → x>>

snd = <p → p <x y → y>>

Properties desired: **fst** (**pair** z_1 z_2) = z
snd (**pair** z_1 z_2) =

fst (**pair** z_1 z_2)
= **fst** (<x y → <s → (s x) y>> z_1 z_2)
= **fst** (<y → <s → (s z_1) y>> z_2)
= **fst** <s → (s z_1) z_2>
= <p → p <x y → x>> <s → (s z_1) z_2>
= <s → (s z_1) z_2> <x y → x>
= (<x y → x> z_1) z_2
= <y → z_1> z_2
= z_1

Encoding pairs

A pair is a data structure holding two val

pair = <x y → <s → (s x) y>>

fst = <p → p <x y → x>>

snd = <p → p <x y → y>>

Properties desired: **fst** (**pair** z_1 z_2) = z

snd (**pair** z_1 z_2) =

snd (**pair** z_1 z_2)

Encoding pairs

A pair is a data structure holding two val

pair = <x y → <s → (s x) y>>

fst = <p → p <x y → x>>

snd = <p → p <x y → y>>

Properties desired: **fst** (**pair** z_1 z_2) = z
snd (**pair** z_1 z_2) =

snd (**pair** z_1 z_2)
= **snd** (<x y → <s → (s x) y>> z_1 z_2)

Encoding pairs

A pair is a data structure holding two val

pair = <x y → <s → (s x) y>>

fst = <p → p <x y → x>>

snd = <p → p <x y → y>>

Properties desired: **fst** (**pair** z_1 z_2) = z
snd (**pair** z_1 z_2) =

snd (**pair** z_1 z_2)
= **snd** (<x y → <s → (s x) y>> z_1 z_2)
= **snd** (<y → <s → (s z_1) y>> z_2)

Encoding pairs

A pair is a data structure holding two val

pair = <x y → <s → (s x) y>>

fst = <p → p <x y → x>>

snd = <p → p <x y → y>>

Properties desired: **fst** (**pair** z_1 z_2) = z
snd (**pair** z_1 z_2) =

snd (**pair** z_1 z_2)
= **snd** (<x y → <s → (s x) y>> z_1 z_2)
= **snd** (<y → <s → (s z_1) y>> z_2)
= **snd** <s → (s z_1) z_2>

Encoding pairs

A pair is a data structure holding two val

pair = <x y → <s → (s x) y>>

fst = <p → p <x y → x>>

snd = <p → p <x y → y>>

Properties desired: **fst** (**pair** z_1 z_2) = z
snd (**pair** z_1 z_2) =

snd (**pair** z_1 z_2)
= **snd** (<x y → <s → (s x) y>> z_1 z_2)
= **snd** (<y → <s → (s z_1) y>> z_2)
= **snd** <s → (s z_1) z_2>
= <p → p <x y → y>> <s → (s z_1) z_2>

Encoding pairs

A pair is a data structure holding two val

pair = <x y → <s → (s x) y>>

fst = <p → p <x y → x>>

snd = <p → p <x y → y>>

Properties desired: **fst** (**pair** z_1 z_2) = z

snd (**pair** z_1 z_2) =

snd (**pair** z_1 z_2)

= **snd** (<x y → <s → (s x) y>> z_1 z_2)

= **snd** (<y → <s → (s z_1) y>> z_2)

= **snd** <s → (s z_1) z_2>

= <p → p <x y → y>> <s → (s z_1) z_2>

= <s → (s z_1) z_2> <x y → y>

Encoding pairs

A pair is a data structure holding two val

pair = <x y → <s → (s x) y>>

fst = <p → p <x y → x>>

snd = <p → p <x y → y>>

Properties desired: **fst** (**pair** z_1 z_2) = z

snd (**pair** z_1 z_2) =

snd (**pair** z_1 z_2)

= **snd** (<x y → <s → (s x) y>> z_1 z_2)

= **snd** (<y → <s → (s z_1) y>> z_2)

= **snd** <s → (s z_1) z_2>

= <p → p <x y → y>> <s → (s z_1) z_2>

= <s → (s z_1) z_2> <x y → y>

= (<x y → y> z_1) z_2

Encoding pairs

A pair is a data structure holding two val

pair = <x y → <s → (s x) y>>

fst = <p → p <x y → x>>

snd = <p → p <x y → y>>

Properties desired: **fst** (**pair** z_1 z_2) = z
snd (**pair** z_1 z_2) =

snd (**pair** z_1 z_2)
= **snd** (<x y → <s → (s x) y>> z_1 z_2)
= **snd** (<y → <s → (s z_1) y>> z_2)
= **snd** <s → (s z_1) z_2>
= <p → p <x y → y>> <s → (s z_1) z_2>
= <s → (s z_1) z_2> <x y → y>
= (<x y → y> z_1) z_2
= <y → y> z_2

Encoding pairs

A pair is a data structure holding two val

pair = <x y → <s → (s x) y>>

fst = <p → p <x y → x>>

snd = <p → p <x y → y>>

Properties desired: **fst** (**pair** z_1 z_2) = z
snd (**pair** z_1 z_2) =

snd (**pair** z_1 z_2)
= **snd** (<x y → <s → (s x) y>> z_1 z_2)
= **snd** (<y → <s → (s z_1) y>> z_2)
= **snd** <s → (s z_1) z_2>
= <p → p <x y → y>> <s → (s z_1) z_2>
= <s → (s z_1) z_2> <x y → y>
= (<x y → y> z_1) z_2
= <y → y> z_2
= z_2

Encoding natural numbers

Church numerals : the encoding of *n* takes a function and applies it *n* times

Encoding natural numbers

Church numerals : the encoding of *n* takes a function and applies it *n* times

0 = <f x → x>

1 = <f x → f x>

2 = <f x → f (f x)>

3 = <f x → f (f (f x))>

…

n = <f x → f (f (... f (f x) …))> (*n* times)

Encoding natural numbers

Church numerals : the encoding of *n* takes a function and applies it *n* times

0 = <f x → x>
1 = <f x → f x>
2 = <f x → f (f x)>
3 = <f x → f (f (f x))>
…
n = <f x → f (f (... f (f x) …))> (*n* times)

succ = <n → <f x → (n f) (f x)>>

zero? = <n → (n <x → **false**>) **true**>

Encoding natural num

Church numerals : the encoding o

0 = <f x → x>
1 = <f x → f x>
2 = <f x → f (f x)>
3 = <f x → f (f (f x))>
…
n = <f x → f (f (... f (f x) …))>

succ = <n → <f x → (n f) (f x)>>

zero? = <n → (n <x → **false**>) **tru**

succ 2

Encoding natural num

Church numerals : the encoding o

0 = <f x → x>
1 = <f x → f x>
2 = <f x → f (f x)>
3 = <f x → f (f (f x))>
…
n = <f x → f (f (... f (f x) …))>

succ = <n → <f x → (n f) (f x)>>

zero? = <n → (n <x → **false**>) **tru**

succ 2
= <n → <f x → (n f) (f x)>> **2**

Encoding natural num

Church numerals : the encoding o

0 = <f x → x>
1 = <f x → f x>
2 = <f x → f (f x)>
3 = <f x → f (f (f x))>
…
n = <f x → f (f (... f (f x) …))>

succ = <n → <f x → (n f) (f x)>>

zero? = <n → (n <x → **false**>) **tru**

succ 2
= <n → <f x → (n f) (f x)>> **2**
= <f x → (**2** f) (f x)>

Encoding natural num

Church numerals : the encoding o

0 = <f x → x>
1 = <f x → f x>
2 = <f x → f (f x)>
3 = <f x → f (f (f x))>
…
n = <f x → f (f (... f (f x) …))>

succ = <n → <f x → (n f) (f x)>>

zero? = <n → (n <x → **false**>) **tru**

succ 2
= <n → <f x → (n f) (f x)>> **2**
= <f x → (**2** f) (f x)>
= <f x → (<f x → f (f x)> f) (f x)>

Encoding natural num

Church numerals : the encoding o

0 = <f x → x>
1 = <f x → f x>
2 = <f x → f (f x)>
3 = <f x → f (f (f x))>
…
n = <f x → f (f (... f (f x) …))>

succ = <n → <f x → (n f) (f x)>>

zero? = <n → (n <x → **false**>) **tru**

succ 2
= <n → <f x → (n f) (f x)>> **2**
= <f x → (**2** f) (f x)>
= <f x → (<f x → f (f x)> f) (f x)>
= <f x → <x → f (f x)> (f x)>

Encoding natural num

Church numerals : the encoding o

0 = <f x → x>
1 = <f x → f x>
2 = <f x → f (f x)>
3 = <f x → f (f (f x))>
…
n = <f x → f (f (... f (f x) …))>

succ = <n → <f x → (n f) (f x)>>

zero? = <n → (n <x → **false**>) **tru**

succ 2
= <n → <f x → (n f) (f x)>> **2**
= <f x → (**2** f) (f x)>
= <f x → (<f x → f (f x)> f) (f x)>
= <f x → <x → f (f x)> (f x)>
= <f x → f (f (f x))>

Encoding natural num

Church numerals : the encoding o

0 = <f x → x>
1 = <f x → f x>
2 = <f x → f (f x)>
3 = <f x → f (f (f x))>
…
n = <f x → f (f (... f (f x) …))>

succ = <n → <f x → (n f) (f x)>>

zero? = <n → (n <x → **false**>) **tru**

succ 2
= <n → <f x → (n f) (f x)>> **2**
= <f x → (**2** f) (f x)>
= <f x → (<f x → f (f x)> f) (f x)>
= <f x → <x → f (f x)> (f x)>
= <f x → f (f (f x))>
= **3**

Encoding natural num

Church numerals : the encoding o

0 = <f x → x>
1 = <f x → f x>
2 = <f x → f (f x)>
3 = <f x → f (f (f x))>
…
n = <f x → f (f (... f (f x) …))>

succ = <n → <f x → (n f) (f x)>>

zero? = <n → (n <x → **false**>) **tru**

zero? 0

Encoding natural num

Church numerals : the encoding o

0 = <f x → x>
1 = <f x → f x>
2 = <f x → f (f x)>
3 = <f x → f (f (f x))>
…
n = <f x → f (f (... f (f x) …))>

succ = <n → <f x → (n f) (f x)>>

zero? = <n → (n <x → **false**>) **tru**

zero? 0
= <n → (n <x → **false**>) **true**> **0**

Encoding natural num

Church numerals : the encoding o

0 = <f x → x>
1 = <f x → f x>
2 = <f x → f (f x)>
3 = <f x → f (f (f x))>
…
n = <f x → f (f (... f (f x) …))>

succ = <n → <f x → (n f) (f x)>>

zero? = <n → (n <x → **false**>) **tru**

zero? 0
= <n → (n <x → **false**>) **true**> **0**
= (**0** <x → **false**>) **true**

Encoding natural num

Church numerals : the encoding o

0 = <f x → x>
1 = <f x → f x>
2 = <f x → f (f x)>
3 = <f x → f (f (f x))>
…
n = <f x → f (f (... f (f x) …))>

succ = <n → <f x → (n f) (f x)>>

zero? = <n → (n <x → **false**>) **tru**

zero? 0
= <n → (n <x → **false**>) **true**> **0**
= (**0** <x → **false**>) **true**
= (<f x → x> <x → **false**>) **true**

Encoding natural num

Church numerals : the encoding o

0 = <f x → x>
1 = <f x → f x>
2 = <f x → f (f x)>
3 = <f x → f (f (f x))>
…
n = <f x → f (f (... f (f x) …))>

succ = <n → <f x → (n f) (f x)>>

zero? = <n → (n <x → **false**>) **tru**

zero? 0
= <n → (n <x → **false**>) **true**> **0**
= (**0** <x → **false**>) **true**
= (<f x → x> <x → **false**>) **true**
= <x → x> **true**

Encoding natural num

Church numerals : the encoding o

0 = <f x → x>
1 = <f x → f x>
2 = <f x → f (f x)>
3 = <f x → f (f (f x))>
…
n = <f x → f (f (... f (f x) …))>

succ = <n → <f x → (n f) (f x)>>

zero? = <n → (n <x → **false**>) **tru**

zero? 0
= <n → (n <x → **false**>) **true**> **0**
= (**0** <x → **false**>) **true**
= (<f x → x> <x → **false**>) **true**
= <x → x> **true**
= **true**

Encoding natural num

Church numerals : the encoding o

0 = <f x → x>
1 = <f x → f x>
2 = <f x → f (f x)>
3 = <f x → f (f (f x))>
…
n = <f x → f (f (... f (f x) …))>

succ = <n → <f x → (n f) (f x)>>

zero? = <n → (n <x → **false**>) **tru**

zero? 2

Encoding natural num

Church numerals : the encoding o

0 = <f x → x>
1 = <f x → f x>
2 = <f x → f (f x)>
3 = <f x → f (f (f x))>
…
n = <f x → f (f (... f (f x) …))>

succ = <n → <f x → (n f) (f x)>>

zero? = <n → (n <x → **false**>) **tru**

zero? 2
= <n → (n <x → **false**>) **true**> **2**

Encoding natural num

Church numerals : the encoding o

0 = <f x → x>
1 = <f x → f x>
2 = <f x → f (f x)>
3 = <f x → f (f (f x))>
…
n = <f x → f (f (... f (f x) …))>

succ = <n → <f x → (n f) (f x)>>

zero? = <n → (n <x → **false**>) **tru**

zero? 2
= <n → (n <x → **false**>) **true**> **2**
= (**2** <x → **false**>) **true**

Encoding natural num

Church numerals : the encoding o

0 = <f x → x>
1 = <f x → f x>
2 = <f x → f (f x)>
3 = <f x → f (f (f x))>
…
n = <f x → f (f (... f (f x) …))>

succ = <n → <f x → (n f) (f x)>>

zero? = <n → (n <x → **false**>) **tru**

zero? 2
= <n → (n <x → **false**>) **true**> **2**
= (**2** <x → **false**>) **true**
= (<f x → f (f x)> <x → **false**>) **true**

Encoding natural num

Church numerals : the encoding o

0 = <f x → x>

1 = <f x → f x>

2 = <f x → f (f x)>

3 = <f x → f (f (f x))>

…

n = <f x → f (f (... f (f x) …))>

succ = <n → <f x → (n f) (f x)>>

zero? = <n → (n <x → **false**>) **tru**

zero? 2

= <n → (n <x → **false**>) **true**> **2**

= (**2** <x → **false**>) **true**

= (<f x → f (f x)> <x → **false**>) **true**

= <x → <x → **false**> (<x → **false**> x)> **true**

Encoding natural num

Church numerals : the encoding o

0 = <f x → x>
1 = <f x → f x>
2 = <f x → f (f x)>
3 = <f x → f (f (f x))>
…
n = <f x → f (f (... f (f x) …))>

succ = <n → <f x → (n f) (f x)>>

zero? = <n → (n <x → **false**>) **tru**

zero? 2
= <n → (n <x → **false**>) **true**> **2**
= (**2** <x → **false**>) **true**
= (<f x → f (f x)> <x → **false**>) **true**
= <x → <x → **false**> (<x → **false**> x)> **true**
= <x → **false**> (<x → **false**> **true**)

Encoding natural num

Church numerals : the encoding o

0 = <f x → x>
1 = <f x → f x>
2 = <f x → f (f x)>
3 = <f x → f (f (f x))>
…
n = <f x → f (f (... f (f x) …))>

succ = <n → <f x → (n f) (f x)>>

zero? = <n → (n <x → **false**>) **tru**

zero? 2
= <n → (n <x → **false**>) **true**> **2**
= (**2** <x → **false**>) **true**
= (<f x → f (f x)> <x → **false**>) **true**
= <x → <x → **false**> (<x → **false**> x)> **true**
= <x → **false**> (<x → **false**> **true**)
= **false**

Arithmetic

plus = <m n → (m **succ)** n>

times = <m n → (m (**plus** n)) **0**>

Arithmetic

plus = <m n → (m **succ)** n>

times = <m n → (m (**plus** n)) **0**

plus 2 3

Arithmetic

plus = <m n → (m **succ)** n>

times = <m n → (m (**plus** n)) **0**

plus 2 3

= <m n → (m **succ)** n> **2 3**

Arithmetic

plus = <m n → (m **succ)** n>

times = <m n → (m (**plus** n)) **0**

plus 2 3

= <m n → (m **succ)** n> **2 3**

= <n → (**2 succ**) n> **3**

Arithmetic

plus = <m n → (m **succ)** n>

times = <m n → (m (**plus** n)) **0**

plus 2 3

= <m n → (m **succ)** n> **2 3**

= <n → (**2 succ**) n> **3**

= (**2 succ**) **3**

Arithmetic

plus = <m n → (m **succ)** n>

times = <m n → (m (**plus** n)) **0**

plus 2 3

= <m n → (m **succ)** n> **2 3**

= <n → (**2 succ**) n> **3**

= (**2 succ**) **3**

= (<f x → f (f x)> **succ**) **3**

Arithmetic

plus = <m n → (m **succ)** n>

times = <m n → (m (**plus** n)) **0**

plus 2 3

= <m n → (m **succ)** n> **2 3**

= <n → (**2 succ**) n> **3**

= (**2 succ**) **3**

= (<f x → f (f x)> **succ**) **3**

= <x → **succ** (**succ** x)> **3**

Arithmetic

plus = <m n → (m **succ)** n>

times = <m n → (m (**plus** n)) **0**

plus 2 3

= <m n → (m **succ)** n> **2 3**

= <n → (**2 succ**) n> **3**

= (**2 succ**) **3**

= (<f x → f (f x)> **succ**) **3**

= <x → **succ** (**succ** x)> **3**

= **succ** (**succ 3**)

Arithmetic

plus = <m n → (m **succ)** n>

times = <m n → (m (**plus** n)) **0**

plus 2 3

= <m n → (m **succ)** n> **2 3**

= <n → (**2 succ**) n> **3**

= (**2 succ**) **3**

= (<f x → f (f x)> **succ**) **3**

= <x → **succ** (**succ** x)> **3**

= **succ** (**succ 3**)

= …

= **5**

Arithmetic

plus = <m n → (m **succ)** n>

times = <m n → (m (**plus** n)) **0**

times 2 3

Arithmetic

plus = <m n → (m **succ)** n>

times = <m n → (m (**plus** n)) **0**

times 2 3

= <m n → (m (**plus** n)) **0**> **2 3**

Arithmetic

plus = <m n → (m **succ)** n>

times = <m n → (m (**plus** n)) **0**

times 2 3

= <m n → (m (**plus** n)) **0**> **2 3**

= <n → (**2** (**plus** n)) **0**> **3**

Arithmetic

plus = <m n → (m **succ)** n>

times = <m n → (m (**plus** n)) **0**

times 2 3

= <m n → (m (**plus** n)) **0**> **2 3**

= <n → (**2** (**plus** n)) **0**> **3**

= (**2** (**plus 3**)) **0**

Arithmetic

plus = <m n → (m **succ)** n>

times = <m n → (m (**plus** n)) **0**

times 2 3

= <m n → (m (**plus** n)) **0**> **2 3**

= <n → (**2** (**plus** n)) **0**> **3**

= (**2** (**plus 3**)) **0**

= (<f x → f (f x)> (**plus 3**)) **0**

Arithmetic

plus = <m n → (m **succ)** n>

times = <m n → (m (**plus** n)) **0**

times 2 3

= <m n → (m (**plus** n)) **0**> **2 3**

= <n → (**2** (**plus** n)) **0**> **3**

= (**2** (**plus 3**)) **0**

= (<f x → f (f x)> (**plus 3**)) **0**

= <x → (**plus 3**) ((**plus 3**) x)> **0**

Arithmetic

plus = <m n → (m **succ**) n>

times = <m n → (m (**plus** n)) **0**

times 2 3

= <m n → (m (**plus** n)) **0**> **2 3**

= <n → (**2** (**plus** n)) **0**> **3**

= (**2** (**plus 3**)) **0**

= (<f x → f (f x)> (**plus 3**)) **0**

= <x → (**plus 3**) ((**plus 3**) x)> **0**

= (**plus 3**) ((**plus 3**) **0**)

Arithmetic

plus = <m n → (m **succ)** n>

times = <m n → (m (**plus** n)) **0**

times 2 3

= <m n → (m (**plus** n)) **0**> **2 3**

= <n → (**2** (**plus** n)) **0**> **3**

= (**2** (**plus 3**)) **0**

= (<f x → f (f x)> (**plus 3**)) **0**

= <x → (**plus 3**) ((**plus 3**) x)> **0**

= (**plus 3**) ((**plus 3**) **0**)

= **plus 3** (**plus 3 0**)

Arithmetic

plus = <m n → (m **succ**) n>

times = <m n → (m (**plus** n)) **0**

times 2 3

= <m n → (m (**plus** n)) **0**> **2 3**

= <n → (**2** (**plus** n)) **0**> **3**

= (**2** (**plus 3**)) **0**

= (<f x → f (f x)> (**plus 3**)) **0**

= <x → (**plus 3**) ((**plus 3**) x)> **0**

= (**plus 3**) ((**plus 3**) **0**)

= **plus 3** (**plus 3 0**)

= …

= **6**

Arithmetic

plus = <m n → (m **succ**) n>

times = <m n → (m (**plus** n)) **0**>

pred = <n → ??? >

Arithmetic

plus = <m n → (m **succ**) n>

times = <m n → (m (**plus** n)) **0**>

pred = <n → **snd** (n **F** (**pair 0 0**))>
where **F** = <p → **pair** (**succ** (**fst** p)) (**fst** p))>

Key property: **F** (**pair** *i j*) = **pair** (*i* + *1*, *i*)

So *n* **F** (**pair 0 0**) = **F** (**F** (... **F** (**pair 0 0**) …)) = **pair** (*n*, *n - 1*)
and **snd** (*n* **F** (**pair 0 0**)) = *n - 1*